

CONNECTING THE DOTS: LOG ANALYSIS



16 May 2026



AGENDA

-  1. Introduction to Logs
-  2. Log Standards (RFC)
-  3. Log Server & Transport
-  4. Log Categories & Formats
-  5. Logs & Telemetry
-  6. Log Deep Dive
-  7. Exercise
-  8. Logs at Scale: SIEM & Data Lake



1. INTRODUCTION TO LOGS

- WHAT IS A LOG? WHY DOES IT EXIST?

WHAT IS A LOG?

A log is a **timestamped, structured record** of an event — a machine writing things down.



Laptop / PC

Auth events, app crashes,
kernel messages



Mobile phone

App logs, crash reports,
battery events



Web server

Every HTTP request —
method, path, status



SSH daemon

Every login attempt — success
and failure



Mail server

Every email sent, received, or
rejected



Router / FW

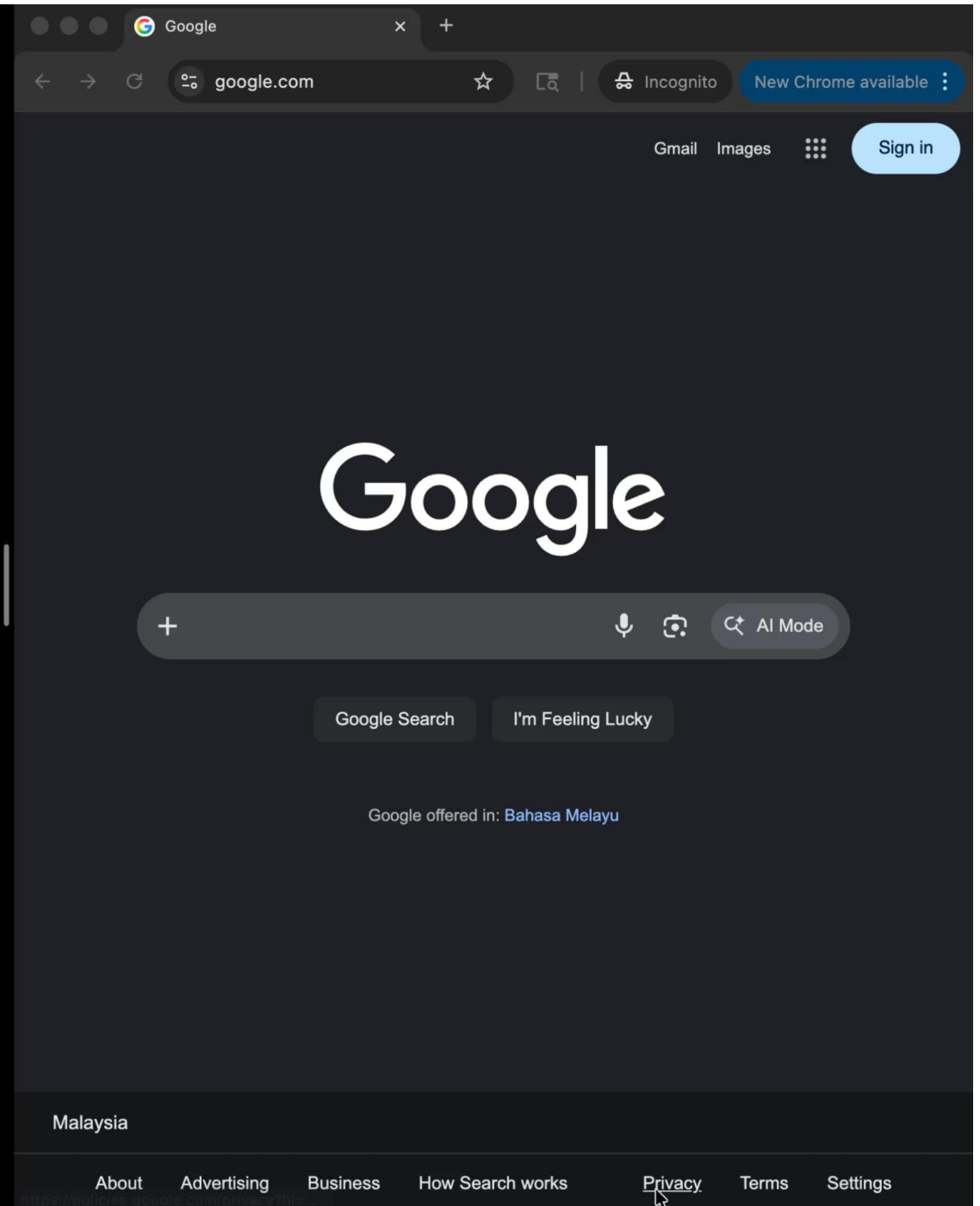
Every connection — who
talked to whom

```
(xⓈ x)-[~]  
$ sudo tail -f /var/log/apache2/access.log
```

```
(xⓈ x)-[~]  
$ sudo tail -f /var/log/apache2/error.log
```

5

```
[0] 0:zsh* "x" 02:57 12-May-26
```



```
(x⊗x)-[~]  
$ sudo tail -f /var/log/auth.log
```

```
x@dark ~ %
```



2. LOG STANDARDS (RFC)

- RFC 3164, RFC 5424, RFC 5425

SYSLOG STANDARDS — NOT RANDOM, THERE IS A SPEC

RFC 3164

BSD Syslog (1984)

Legacy format. Priority byte, hostname, message.
Timestamp has NO year. Still common on older Linux.

RFC 5424

Modern Syslog

Adds: VERSION, structured data <key=value>,
MSG-ID, APP-NAME. ISO 8601 timestamp with timezone.

RFC 5425

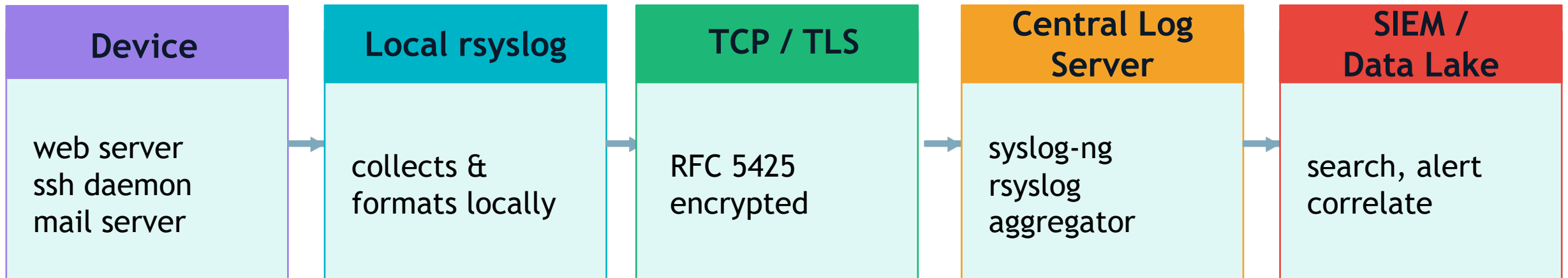
Syslog over TLS

Transport security for log shipping across networks.
Use this when sending logs off-machine.

RFC 5424 line anatomy:

```
<34>1      2026-05-06T08:11:08+08:00  webserver01  sshd  2366  -  -  Failed password for root from 203.0.113.9
PRI  VER  TIMESTAMP                HOSTNAME    APP      PID  MSGID SD  MESSAGE
```


3. LOG SERVER & TRANSPORT



UDP 514 — legacy, no delivery guarantee **TCP** — reliable **TLS/TCP (RFC 5425)** — encrypted + reliable ← use this

`logrotate` — prevents disk exhaustion. Configured in `/etc/logrotate.conf`. Rotate daily, keep 30 days, compress.



4. LOG CATEGORIES & FORMATS

- NETWORK
- APPLICATION
- SYSTEM
- ACCESS
- SECURITY/AUDIT

THE FIVE LOG CATEGORIES



Network

Firewall, IDS, router

Format: CEF / IPFIX / NetFlow

Connections, packets, flows



Application

Apache, MySQL, SMTP,
custom app

Format: CLF / W3C / JSON / plain text

Requests, queries, errors



System

OS kernel, services

Format: Syslog RFC 5424

Boot, shutdown, daemon events



Access / Audit

SSH, auditd

Format: Syslog / audit

Login attempts, privilege use



Security / Audit

Windows Security Event
Log, auditd

Format: EVTX / syslog

Policy change, account mgmt, logon (Event ID 4625)

5. LOGS & TELEMETRY — THE THREE PILLARS OF OBSERVABILITY



Logs

Discrete events with context.
'X happened at time T on host Y.'
Structured or plain text.

e.g. `auth.log`,
`apache_access.log`



Metrics

Numeric measurements over time.
CPU at 82%. 450 req/sec. Disk 73%.
Time-series database.

e.g. Prometheus,
InfluxDB, CloudWatch



Traces

Request journey across services.
Used in microservices / distributed systems. Span-based.

e.g. Jaeger, Zipkin,
OpenTelemetry



6. LOG DEEP DIVE

SSH · APACHE · MYSQL · DNS · SMTP · PCAP · WINDOWS/LINUX

6.1 SSH AUTH LOG — HOST / ACCESS LOG

```
May 6 08:11:08 webserver01 sshd[2366]: Failed password for root from 203.0.113.9 port 41657 ssh2
May 6 08:11:11 webserver01 sshd[2391]: Failed password for root from 203.0.113.9 port 57713 ssh2
May 6 08:11:12 webserver01 sshd[2318]: Failed password for root from 203.0.113.9 port 43056 ssh2
May 6 08:10:01 webserver01 sshd[2312]: Accepted password for miko from 192.168.1.45 port 42470 ssh2
```

Timestamp

May 6 08:11:08

Hostname

webserver01

Process+PID

sshd[2366]

Outcome

Failed / Accepted

Username

root / miko

Source IP

203.0.113.9

⚠ Same IP · same user · 3 consecutive failures in 4 seconds = brute-force pattern

6.2 APACHE ACCESS LOG — APPLICATION LOG

```
203.0.113.9 - - [06/May/2026:08:07:16 +0800] "GET /admin HTTP/1.1" 404 217 "-" "Nikto/2.1.6"  
192.168.1.45- - [06/May/2026:08:03:38 +0800] "GET /slow-query HTTP/1.1" 200 90166 "-" "python-requests/2.31.0"  
10.0.0.23 - - [06/May/2026:08:02:52 +0800] "POST /login HTTP/1.1" 200 303 "-" "Mozilla/5.0 ..."
```

Client IP

203.0.113.9

Timestamp

08:07:16 +0800

Method

GET / POST

Path

/admin

Status code

200 · 302 · 404

Bytes

90166 bytes

User-Agent "Nikto/2.1.6" → automated scanner. Legitimate browsers never identify as Nikto.

6.3 MYSQL LOGS — APPLICATION LOG

```
2026-05-06T08:15:32.123456Z 14 Query      SELECT * FROM users WHERE id=1001
2026-05-06T08:15:33.004521Z 14 Query      SELECT password FROM users WHERE username='admin'
2026-05-06T08:22:11.441200Z  2 Connect    Access denied for user 'root'@'203.0.113.9' (using password: YES)
/var/log/mysql/error.log:
2026-05-06T08:22:11Z [ERROR] Access denied for user 'root'@'203.0.113.9'
```

MySQL produces three separate log files:

General query log

Every SQL statement. Username · timestamp · query text. Very verbose.

Error log

Startup/shutdown, connection errors, access denied.

Slow query log

Queries exceeding long_query_time. Performance hunting.

6.4 DNS / DHCP LOGS — SERVICE / INFRASTRUCTURE LOG

```
06-May-2026 08:18:32.441 queries: client 192.168.1.45#53210: query: evil.example.com IN A + (10.0.0.1)
06-May-2026 08:18:32.999 queries: client 192.168.1.45#53211: query: evil.example.com IN A NXDOMAIN
May  6 08:05:00 dhcpcd: DHCPREQUEST for 192.168.1.45 from aa:bb:cc:dd:ee:ff (laptop01) via eth0
May  6 08:05:00 dhcpcd: DHCPACK on 192.168.1.45 to aa:bb:cc:dd:ee:ff (laptop01) via eth0
```

Key DNS fields to know:

NXDOMAIN

Non-existent domain response — domain does not exist. Common in malware C2 callback failures and DGA detection.

Query type

A (IPv4), AAAA (IPv6), MX (mail), TXT (SPF/DKIM), PTR (reverse lookup)

Client IP

The machine that asked the question — trace who looked up what

6.5 SMTP (POSTFIX) LOG — APPLICATION LOG

```
May 6 08:05:01 mail01 postfix/smtpd[4407]: GHC0CLBR7T: client=mail.example.com[10.0.0.23]  
May 6 08:05:03 mail01 postfix/smtp[4510]: GHC0CLBR7T: to=<admin@corp.local>, status=sent  
May 6 08:21:01 mail01 postfix/smtpd[4481]: NOQUEUE: reject: RCPT from unknown[203.0.113.9]:  
554 5.7.1 Relay access denied; from=<spam@evil.example>
```

Queue ID

GHC0CLBR7T

Tracks one email through the full Postfix pipeline — all lines for one email share this ID

status=sent

Delivery outcome

sent · bounced · deferred · reject — this tells you if the email was delivered

NOQUEUE: reject

Pre-queue block

The connection was refused before Postfix even queued it — 554 5.7.1 = relay denied

from= / to=

Sender & recipient

Who sent it and who it was addressed to — key for mail security investigations

6.6 HOST LOGS — WINDOWS EVENT LOG & LINUX SYSLOG

Windows

```
EventID 4625 — Failed logon: Account=Administrator  
Src=203.0.113.9 LogonType=3  
EventID 4624 — Successful logon: Account=miko  
Src=192.168.1.45 LogonType=3  
EventID 4688 — New process: calc.exe spawned by cmd.exe  
User=SYSTEM
```

Key Windows Event IDs:

4624

Successful
logon

4625

Failed logon

4648

Explicit
credential

4688

New process
created

Linux

```
/var/log/syslog: May 6 08:22:11 host kernel: segfault  
at 00 ip 7f...  
/var/log/auth.log: Accepted publickey for miko from  
192.168.1.45  
auditd: type=SYSCALL msg=... syscall=59 comm=bash  
uid=1000 auid=1000
```

Linux audit (auditd):

syscall: Which kernel call was made (59=execve = program run)

uid / auid: Real user and audit user — who really ran it

comm: Command name — what process triggered the call

6.7 PCAP — NETWORK CAPTURE (NOT A LOG)

PCAP is NOT a log in the RFC sense — it is a raw binary packet capture. What the wire actually said.

PCAP vs Log:

Type	Format	Tool	Contains
PCAP	Raw binary (libpcap format)	tcpdump, Wireshark, scapy	Every byte on the wire — payload included
Log	Structured text / syslog	grep, Python, SIEM	What the application chose to record



7. EXERCISE

- HANDS-ON: 5 TASKS ACROSS 3 LOG FILES

EXERCISE — 5 TASKS ACROSS 4 LOG FILES

T1

`apache_access.log`

Count lines with status 404. List every IP that caused one.

T2

`apache_access.log`

Find the request with the largest response (bytes). What was the URL?

T3

`auth.log`

Count failed SSH logins per IP. Which IP had the most?

T4

`mail.log`

Find all NOQUEUE reject events. Which IP was rejected and why?

T5

`auth.log + mail.log`

Does any IP appear in both? Write 2 sentences: what was it doing?

Bonus — B1: DNS NXDOMAIN domain name · B2: Scanner User-Agent identification

Connecting the Dots: Log Analysis Exercise



http://tiny.cc/logs_analysis



8. LOGS AT SCALE: SIEM & DATA LAKE

- ELASTIC, SPLUNK, KAFKA — THE BIGGER PICTURE

FROM MANUAL PARSING TO SIEM

Today you read logs manually. In a real SOC there are millions of lines per hour from hundreds of sources.

Elastic Security

Open source SIEM built on Elasticsearch.
KQL / ES|QL query language.
Common in Malaysian enterprise & gov.

Splunk

Enterprise SIEM. SPL query language.
High adoption in large corporations.

Wazuh

Open source. Great for learning.
Free VM available. Start here.

Microsoft Sentinel

Cloud-native SIEM on Azure.
KQL queries. Integrates with M365.

KEY TAKEAWAYS



Logs are everywhere

Every device, OS, and service writes logs.
Always.



Standards exist

RFC 5424 defines syslog. CLF defines Apache.
There is a spec for everything.



SIEM does this at scale

You did it manually today. SIEM does it across
millions of events in real time.



Structure is universal

Timestamp · source · level · message — across
every format and vendor.



Correlation is the superpower

One log tells a partial story. Three logs together
reveal the full incident.



Your next step

Learn KQL or ES|QL. Same log fields, different
query syntax. Start with Elastisc or Wazuh.



"EVERY ALERT IN A SOC STARTS AS A LINE IN A LOG FILE."

You now know what that line looks like.

Further Reading

- RFC 5424 · tools.ietf.org/html/rfc5424
- Wazuh (free SIEM lab) · wazuh.com
- TryHackMe: 'Intro to Logs' · 'Splunk 101' · 'Windows Event Logs'
- HackTheBox Academy: 'Web Request' · 'Security Monitoring & SIEM Fundamentals' · 'Network Foundations'
- ECS reference · elastic.co/guide/en/ecs

THANK YOU

AMIR | amir@eclogic.io

SYAKIR | syakir@eclogic.io

HALIM | halim@eclogic.io

MASLAN | maslan@eclogic.io

AIDA | nur.aida@eclogic.io



Connecting the Dots: Log Analysis
Feedback Form



http://tiny.cc/logs_analysis_feedback

| www.eclogic.io |